SERVICIO NACIONAL DE APRENDIZAJE – SENA
06 de Noviembre de 2026

RaízVolt

Integración de módulos web

Aprendiz:

Jhon Edwards Tineo Landaeta

Programa de formación: Análisis y desarrollo de software (3235874)

Complejo Tecnológico Minero Agroempresarial

- CTMA Regional Antioquia

Instructor:

Diego Armando Barrera Araujo

Actividad de Aprendizaje:

GA8-220501096-AA1. Integrar módulos, de acuerdo con el diseño establecido.

Evidencia de Desempeño:

Desarrollar software a partir de la integración de sus módulos componentes. GA8-220501096-AA1-EV01.

IMPLEMENTACIÓN	VERIFICACIÓN	ENTREGABLES
7 áreas funcionales Node.js + SQLite	85 pruebas Node 13 comprobaciones web	PDF · fuentes · dist/ evidencias · Git

6 de noviembre de 2026 · Versión de la aplicación 2.0.0

Documento de implementación, arquitectura, seguridad, ambientes, pruebas y guía de ejecución.

RAÍZVOLT / INFORME TÉCNICO / 02

Propósito y alcance

Introducción

RaízVolt es un proyecto académico de comercialización de soluciones solares. Esta evidencia integra la autenticación, el perfil del cliente, el catálogo, el inventario, las cotizaciones y los pedidos en una aplicación orientada a la web. Las vistas consumen una API HTTP y comparten una base SQLite persistente.

Se tomó como base el código de AA5-EV03 conservado en la entrega AA5-EV04. La API ya implementaba reglas comerciales; la presente versión añade la interfaz web, componentes reutilizables, compilación, pruebas unitarias y recorridos de integración en navegador. Se revisó el instrumento de evaluación de GA8-AA1-EV01.

Objetivo general

Codificar e integrar los módulos de RaízVolt de acuerdo con sus requisitos, utilizando separación de responsabilidades, controles de seguridad, pruebas reproducibles y control de versiones.

Objetivos específicos

- Relacionar historias de usuario y casos de uso con componentes y pruebas.
- Conectar las pantallas con los servicios de autenticación, catálogo, inventario y ventas.
- Mantener consistencia entre cotización, pedido y existencias mediante transacciones.
- Documentar la arquitectura, las dependencias, los ambientes y el uso del IDE.
- Entregar fuentes y archivos empaquetados que se puedan ejecutar con Node.js 24.x.

Continuidad y límites

Se conserva el contrato Node.js/SQLite de la API anterior. La interfaz React/Spring Boot de entregas previas no se reutiliza en esta versión y no se migra automáticamente su base H2. Los precios, clientes y pedidos usados en las pruebas son ficticios. El alcance no incluye pagos, impuestos, fletes, facturación ni logística real.

Metodología

Se aplica un trabajo incremental con un flujo Kanban: pendiente, desarrollo, pruebas y terminado. Los incrementos cubren recuperación de requisitos, interfaz, integración, verificación y entrega. Un módulo se considera terminado cuando tiene implementación, prueba aprobada, documentación y registro de cambios. No se atribuyen reuniones ni aprobaciones inexistentes a un equipo externo.

RAÍZVOLT / INFORME TÉCNICO / 03

Requisitos y trazabilidad

Las historias HU-PR y HU-API conservan los identificadores de la documentación anterior; las extensiones API se mantienen como propuestas del proyecto.

ID / HISTORIA	CRITERIO DE ACEPTACIÓN	CÓDIGO / PRUEBA
RF01 · Acceso AA5-EV01 / HU-API01	Registro CLIENTE, autenticación, perfil y cierre. No se exponen hashes ni contraseñas.	auth-service.js, account.js U-A01 a U-A06; E-02/E-03
RF02 · HU-PR02	Consultar, buscar, filtrar y paginar productos activos.	catalog-service.js, catalog.js U-C01; E-01/E-11
RF03 · HU-PR01	ADMIN crea un producto con código único, precio válido y stock entero.	CatalogService.save U-V05; E-08
RF04 · HU-PR03	ADMIN edita datos. El stock cambia mediante ajustes con motivo.	admin.js; save / adjust U-C03; E-08
RF05 · HU-PR04	Retirar del catálogo conserva el historial comercial.	CatalogService.remove E-09; pruebas de API
RF06 · HU-API02	Saldo no negativo; movimientos auditables y resumen exacto.	catalog-service.js U-C02/U-C03; E-08
RF07 · HU-API03	Cotizar de 1 a 20 productos; precios del servidor; vigencia de 7 días.	SalesService.createQuote U-S04; E-04
RF08 · HU-API04	Perfil completo, cotización propia y stock; pedido atómico y repetible.	SalesService.createOrder U-S01/U-S05/U-D01; E-04/E-05
RF09 · HU-API05	Cancelar pendientes devuelve stock una vez; ADMIN despacha y entrega.	SalesService.changeState U-S02/U-S03; E-06/E-10

Requisitos no funcionales

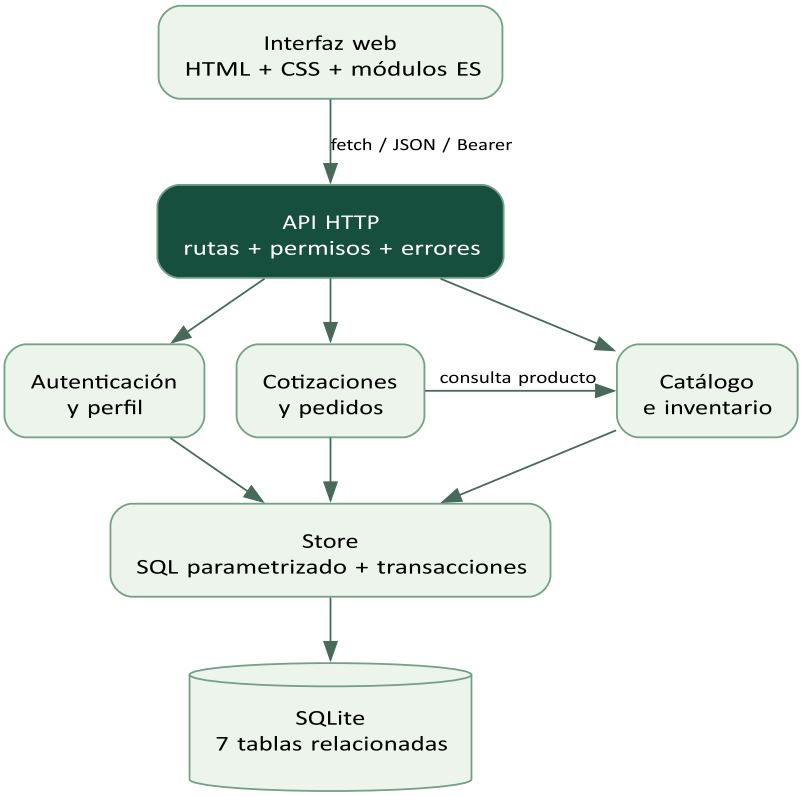
Seguridad: roles y propiedad comprobados en el servidor, validación de datos y sesión temporal. **Mantenibilidad:** módulos ES, nombres descriptivos, separación por responsabilidades y build reproducible. **Usabilidad:** formularios etiquetados, foco visible, mensajes de resultado y adaptación a pantalla móvil.

La matriz ampliada y los casos CU01-CU03 están disponibles en docs/TRAZABILIDAD.md.

RAÍZVOLT / INFORME TÉCNICO / 04

Arquitectura y componentes

Se implementa un monolito modular: los servicios comparten un proceso y una base de datos. La interfaz y la API se sirven desde el mismo origen. La capa HTTP traduce solicitudes en operaciones de los servicios y devuelve resultados o errores controlados.



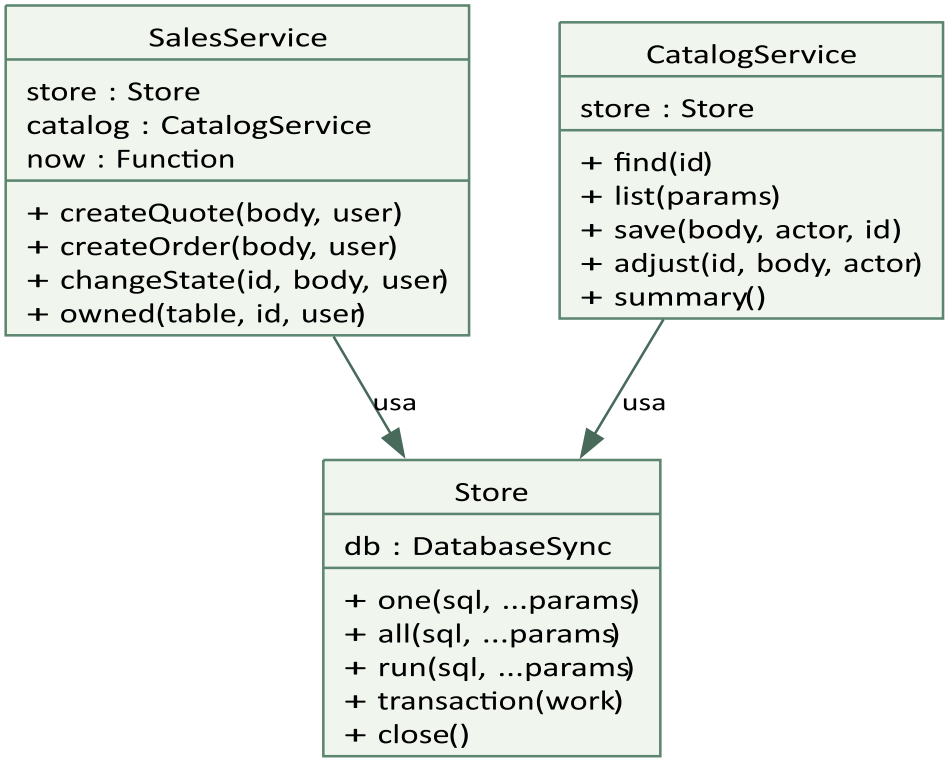
CAPA	TECNOLOGÍA / LIBRERÍA	DECISIÓN
Presentación	HTML5, CSS3, JavaScript ES; fetch y DOM	Sin framework de UI adicional. Componentes pequeños que consumen el contrato existente.
HTTP y seguridad	Node.js 24; node:http y node:crypto	Sin Express ni dependencias de ejecución externas. Rutas, permisos y hash explícitos.
Negocio	Servicios JavaScript ES	Reglas de catálogo, cotización, inventario y pedidos independientes de la vista.
Persistencia	node:sqlite; SQL parametrizado	Base embebida para la demostración, sin instalar un servidor adicional.
Construcción / pruebas	esbuild 0.25.12; node:test; Playwright 1.58.1	Build empaquetado, pruebas unitarias y recorridos en navegador.

Frameworks: no se añade uno nuevo en las capas de ejecución. Esta decisión conserva el backend previo y reduce instalaciones; las librerías de desarrollo están fijadas en package-lock.json.

RAÍZVOLT / INFORME TÉCNICO / 05

Clases y modelo de datos

El diagrama representa clases que existen en el código. Los servicios reciben Store por inyección de dependencias, lo que permite probar comportamientos con dobles de prueba. El reloj de SalesService también se puede sustituir.



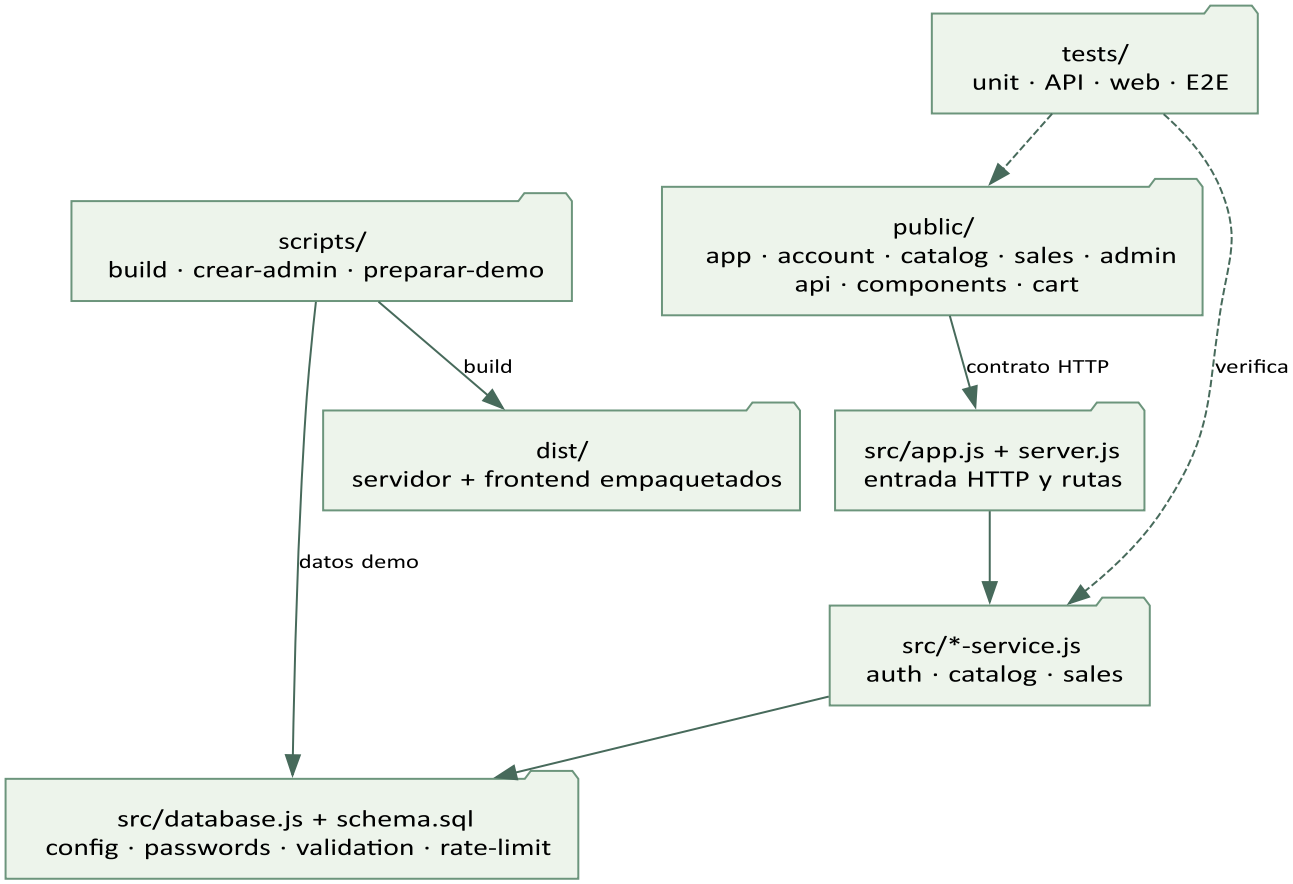
Además, Cart administra la selección en el navegador. createAuthService y createApplication son funciones fábrica, no clases; la primera encapsula autenticación y perfil, y la segunda construye un servidor aislable.

TABLA	RELACIONES Y RESPONSABILIDAD
usuarios / sesiones	Usuario 1:N sesiones. Guarda perfil, rol y hash de contraseña; sesiones guarda hash del token y vencimiento.
productos / movimientos	Producto 1:N movimientos. Código único, precio en centavos, stock y baja lógica; movimiento registra actor, variación, saldo y motivo.
cotizaciones / cotizacion_items	Usuario 1:N cotizaciones; cotización 1:N líneas. Cada línea conserva precio y nombre del producto al cotizar.
pedidos	Usuario 1:N pedidos; cotización 1:0..1 pedido. Conserva estado y copia de datos de entrega. Pedido 1:N movimientos.

Las claves foráneas y restricciones CHECK/UNIQUE se definen en src/schema.sql. La copia de precios y dirección evita que una edición posterior altere documentos comerciales ya registrados.

RAÍZVOLT / INFORME TÉCNICO / 06

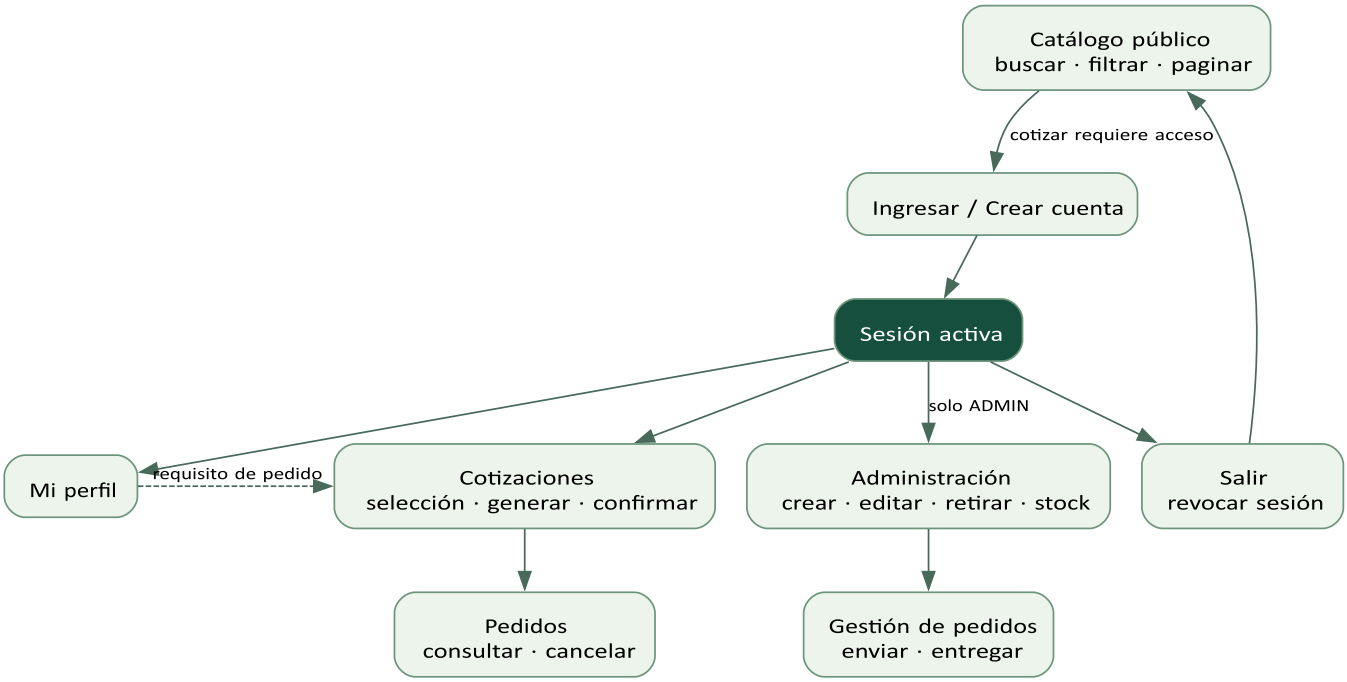
Paquetes y reutilización



PATRÓN / PRÁCTICA	APLICACIÓN CONCRETA
Servicios de aplicación	CatalogService y SalesService concentran reglas; las vistas no calculan precios finales ni autorizan operaciones.
Acceso a datos encapsulado	Store ofrece one, all, run y transaction; concentra la conexión y el manejo de transacciones. No se afirma un ORM inexistente.
Inyección de dependencias	Servicios reciben Store, catálogo y reloj según corresponda. Los tests unitarios sustituyen dependencias.
Componentes reutilizables	api() centraliza HTTP; bindForm y bindAction manejan errores y bloqueo de botones; pagination, escape y currency se comparten.
Separación y buenas prácticas	Módulos ES, camelCase, clases PascalCase, constantes en mayúsculas, UTF-8 y dos espacios. Comentarios explican seguridad, dinero y transacciones.

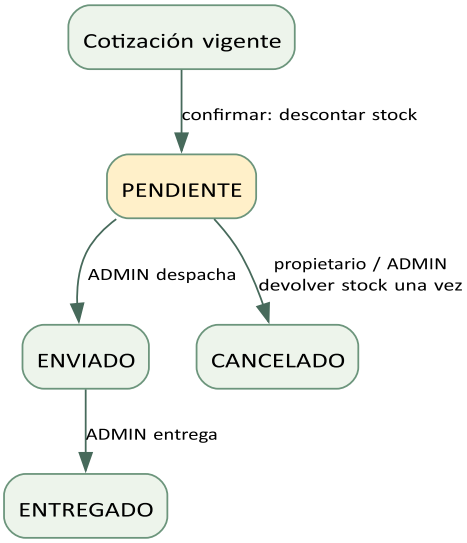
El fuente se agrupa por responsabilidad en public/, src/, scripts/ y tests/. dist/ contiene la salida empaquetada, no el lugar de edición. Los diagramas se entregan también en DOT editable y SVG vectorial.

Navegación e integración



CU01: cotizar y confirmar un pedido

El cliente completa su perfil, selecciona productos, genera una cotización y confirma. El navegador envía IDs y cantidades; el servidor toma precios del catálogo. Al confirmar se verifica propiedad, vigencia, disponibilidad y perfil, y se guardan pedido, descuento de stock y movimientos en una única transacción.



Consistencia: si falta stock en una línea no se confirma ningún descuento parcial. Confirmar de nuevo la misma cotización devuelve el pedido existente. Cancelar un pedido pendiente devuelve inventario una sola vez. No se permite pasar de PENDIENTE directamente a ENTREGADO.

RAÍZVOLT / INFORME TÉCNICO / 08

Mecanismos de seguridad

CONTROL	IMPLEMENTACIÓN	EVIDENCIA
Contraseñas	scrypt con sal individual aleatoria; N=16384, r=8, p=5. Comparación con timingSafeEqual; hash no se devuelve al cliente.	passwords.js U-A01/U-A02
Sesiones	Token aleatorio de 32 bytes; base guarda SHA-256; vence a las 2 horas. Nuevo login revoca el anterior y logout elimina la sesión.	auth-service.js U-A03 a U-A05
Roles y propiedad	Registro público solo CLIENTE. Escrituras de catálogo e inventario solo ADMIN. Documentos filtrados por propietario en servidor.	app.js / owned() U-S01/U-S02; E-07
Entrada y SQL	Lista cerrada de campos, tipos, rangos y longitudes; IDs enteros; consultas con parámetros. Precio decimal convertido a centavos.	validation.js / Store U-V01 a U-V08
Interfaz web	Datos escapados antes de insertarse en HTML; CSP; sin código remoto. Token solo en memoria, no localStorage ni sessionStorage.	components.js / api.js U-F04; E-12
Límites HTTP	Cuerpo de 32 KiB, tiempos máximos y límite de acceso por IP. no-store, nosniff, no-referrer; sin CORS abierto.	app.js / rate-limit.js U-A06; pruebas de API
Archivos y credenciales	Lista cerrada de recursos públicos. No se sirve data/, SQL o .env. ADMIN de demo se genera localmente con contraseña aleatoria.	W-01 preparar-demo.js

Decisiones de autenticación

La autorización se transmite en una cabecera Bearer explícita y no mediante cookies de sesión. El navegador debe iniciar sesión nuevamente al recargar. Los permisos del frontend solo facilitan la navegación: las restricciones efectivas se aplican en el servidor.

Alcance operativo

La aplicación escucha únicamente en 127.0.0.1 para demostración local. Una publicación de producción requiere una configuración específica de HTTPS, proxy, respaldos, operación y recuperación de cuentas. Las pruebas presentadas no constituyen una auditoría de seguridad ni certifican un uso productivo.

RAÍZVOLT / INFORME TÉCNICO / 09

Servidores, base y ambientes

ELEMENTO	DESARROLLO	COMPILADO / PRUEBAS
Runtime	Node.js 24.x	Verificado: Node.js 24.22.1; SQLite 3.53.4.
Arranque	node src/server.js	node dist/src/server.js
Red	127.0.0.1; PORT=3000 por defecto	API: puertos temporales. E2E: 3487; TEST_PORT permite cambiarlo.
Persistencia	data/raizvolt.sqlite	dist/data/raizvolt.sqlite; pruebas con base temporal aislada.
Ruta alternativa	Variable DB_PATH	Permite compartir una base entre fuente y dist.
Herramientas	VS Code, Git, npm	node:test, assert/strict, Playwright y Chromium.

Inicialización de SQLite

Store crea la carpeta y ejecuta schema.sql al iniciar. Activa claves foráneas, WAL y espera de bloqueo de 5000 ms. Las transacciones usan BEGIN IMMEDIATE, COMMIT o ROLLBACK. No se requiere MySQL ni XAMPP. No hay migraciones automáticas desde bases de entregas anteriores.

Ejemplo de configuración en PowerShell

```
$env:PORT = "3001" $env:DB_PATH =
"$PWD/data/compartida.sqlite" npm run
demo:dist npm run start:dist
```

Con esa configuración se abre http://127.0.0.1:3001. Las variables se establecen en la terminal o en el depurador; el proyecto no carga automáticamente un archivo .env. Las rutas por defecto de fuente y dist son distintas.

Respaldo y restauración

Detener el servidor y los procesos que usen SQLite; copiar la carpeta data/ correspondiente completa, incluidos auxiliares si existen. Restaurar con el servidor detenido. No copiar solo un archivo SQLite abierto con WAL activo. Excluir datos, sesiones y credenciales del repositorio.

Condiciones verificadas

Las pruebas y el build se ejecutaron en Linux. Chromium 153.0.8010.0 se probó con escritorio y viewport de 390 × 844. Los .bat y la configuración de VS Code se entregan para reproducción en Windows.

IDE y ejecución paso a paso

Abrir la versión lista para ejecutar en Windows

1. Instalar Node.js 24.x y comprobar **node --version** en una terminal nueva.
2. Extraer completamente el ZIP. Entrar en la carpeta del proyecto.
3. Ejecutar **PREPARAR_DEMO_WINDOWS.bat**. Guardar el usuario ADMIN y la contraseña aleatoria mostrados.
4. Ejecutar **INICIAR_WINDOWS.bat** y mantener abierta la ventana.
5. Abrir http://127.0.0.1:3000. Entrar con ADMIN o crear una cuenta CLIENTE.
6. Completar Mi perfil con datos ficticios. Añadir productos, generar cotización y confirmar pedido. ConADMIN se gestionan catálogo, stock y despacho.
7. Detener el servidor con **Ctrl+C**. Los datos locales se conservan.

dist/ ya está incluido: para esta ejecución no hace falta instalar dependencias npm, compilar ni abrir Postman. Sí se necesita Node.js 24.x.

Trabajar con el código en Visual Studio Code

Usar Archivo > Abrir carpeta, seleccionar el proyecto y abrir Terminal > Nueva terminal. Ejecutar:

```
node --version
npm ci npm run
demo npm start
```

Consultar src/app.js para las rutas, public/app.js para navegación y sales-service.js para pedidos. .vscode/launch.json incluye el perfil **RaizVolt: servidor fuente**; .vscode/tasks.json incluye pruebas y compilación.

Depurar la confirmación de un pedido

Configurar DB_PATH hacia data/desarrollo.sqlite y preparar su demo según docs/AMBIENTES.md.
Detener otros servidores en el puerto 3000. Marcar un breakpoint en SalesService.createOrder, pulsar F5 y confirmar una cotización en el navegador. Inspeccionar Variables, avanzar con F10 y detener la depuración al finalizar.

Estas instrucciones y los archivos de configuración sustentan el uso del IDE.

Construcción y entregables

Compilación reproducible

```
npm ci npm test
npm run build npm
run demo:dist npm
run start:dist
```

esbuild transforma y empaqueta el servidor y las utilidades locales para Node.js 24; agrupa y minifica el frontend para ES2022. Se copian HTML, CSS, esquema SQL y documentación. Los mapas .map permiten relacionar el resultado con las fuentes.

RUTA	CONTENIDO
src/ y public/	Código fuente comentado de servidor e interfaz.
dist/src/server.js	Servidor empaquetado para ejecutar con Node.js.
dist/public/app.js	Frontend compilado y minificado, más HTML y CSS.
dist/scripts/	Preparación de demo y creación local de ADMIN.
dist/BUILD.json	Versión, runtime, herramienta y fecha de construcción.
docs/ y tests/	Informe, contratos, diagramas, ambientes, capturas y pruebas reproducibles.
versionamiento/RaizVolt_GA8.bundl e	Historial Git local transportable, incluido en el ZIP.

En JavaScript, esta entrega de “archivos compilados” corresponde a código transformado y empaquetado. Requiere el runtime Node.js; no es un .exe, .jar ni binario nativo.

Direcciones de ejecución y contrato

Aplicación: http://127.0.0.1:3000
Documentación: http://127.0.0.1:3000/documentacion
OpenAPI: http://127.0.0.1:3000/api/openapi.json

Las rutas locales funcionan después de iniciar el servidor. Las 22 operaciones funcionales de la API se documentan en docs/SERVICIOS.md y docs/openapi.json; incluyen acceso, perfil, productos, inventario, cotizaciones y pedidos.

Pruebas unitarias y de API

La ejecución registrada aprobó **85 casos de Node, sin fallos ni omisiones**. De ellos, 28 son unitarios; el resto verifica la API, una colección de solicitudes y los recursos públicos. Los registros originales están en PRUEBAS.tap y PRUEBAS.xml.

GRUPO	CASOS	VERIFICACIÓN PRINCIPAL
Autenticación y seguridad	6	Sal individual, rechazo de hash corrupto, token, revocación y límite por IP.
Validación	8	Dinero, credenciales, perfil, producto, duplicados, paginación e IDs.
Catálogo e inventario	3	Producto inexistente, resumen monetario y saldo fuera de rango.
Ventas	5	Propiedad, permisos, transiciones, precio del catálogo y vencimiento.
Transacción de Store	1	Confirmar un resultado o revertir una excepción.
Frontend reutilizable	5	Carrito, límites, limpieza, escape HTML y formato monetario.
Integración de API	55	Contratos HTTP, validaciones, autenticación, roles, persistencia y consistencia comercial.
Colección HTTP	1	Recorrido de 34 solicitudes con un ejecutor limitado en Node. No es una ejecución de Postman.
Archivos web	1	HTML/CSS/JS disponibles, CSP y rutas privadas bloqueadas.

Aislamiento y resultados esperados

Las pruebas unitarias ejercitan funciones o servicios con dobles de dependencias y reloj controlado. Las pruebas de integración crean SQLite temporal y un servidor HTTP aislado. No usan la base de la demostración. Se comprueban respuestas válidas y rechazo de operaciones inválidas.

```
npm run test:unit
npm test
```

Ejemplo crítico: al fallar una línea de un pedido, el test verifica que no quede descuento parcial. La repetición de una confirmación se comprueba para impedir pedidos e inventario duplicados. Estos resultados no representan una medición de cobertura porcentual ni una prueba de carga.

Pruebas en navegador

Ejecución sobre dist/ con datos ficticios y una base SQLite temporal. Registro: docs/evidencias/E2E.json.

CASO	RECORRIDO VERIFICADO	RESULTADO
E-01	Catálogo y filtro de búsqueda	Aprobado
E-02	Registro e inicio de sesión de cliente	Aprobado
E-03	Guardado del perfil y datos de entrega	Aprobado
E-04	Cotización y pedido descuentan exactamente dos unidades	Aprobado
E-05	Repetir confirmación no duplica pedido ni descuento	Aprobado
E-06	Cancelación devuelve inventario	Aprobado
E-07	Ruta de administración rechazada para cliente	Aprobado
E-08	Administrador crea, edita y ajusta inventario	Aprobado
E-09	Baja lógica retira el producto del catálogo	Aprobado
E-10	Despacho y entrega con rol administrador	Aprobado
E-11	Catálogo móvil sin desbordamiento horizontal	Aprobado
E-12	Recarga elimina sesión de memoria; no hay tokens en almacenamiento web	Aprobado
E-13	Sin errores JavaScript en los recorridos	Aprobado

Resultado: 13 comprobaciones aprobadas y ningún error JavaScript registrado en los recorridos. Se probó Chromium 153.0.8010.0 con Playwright 1.58.1. La ejecución final quedó registrada el 6 de noviembre de 2026 a las 11:34, hora de Colombia.

```
npm ci npx playwright install
chromium npm run test:e2e
```

Para la ejecución registrada se suministró un Chromium instalado por otra vía mediante CHROME_PATH. El navegador no se incluye en el ZIP. El script crea y elimina su propia base temporal, inicia el servidor compilado y guarda resultados y capturas.

Evidencia visual: catálogo

Captura real de la aplicación compilada. Los precios y productos son datos de prueba.

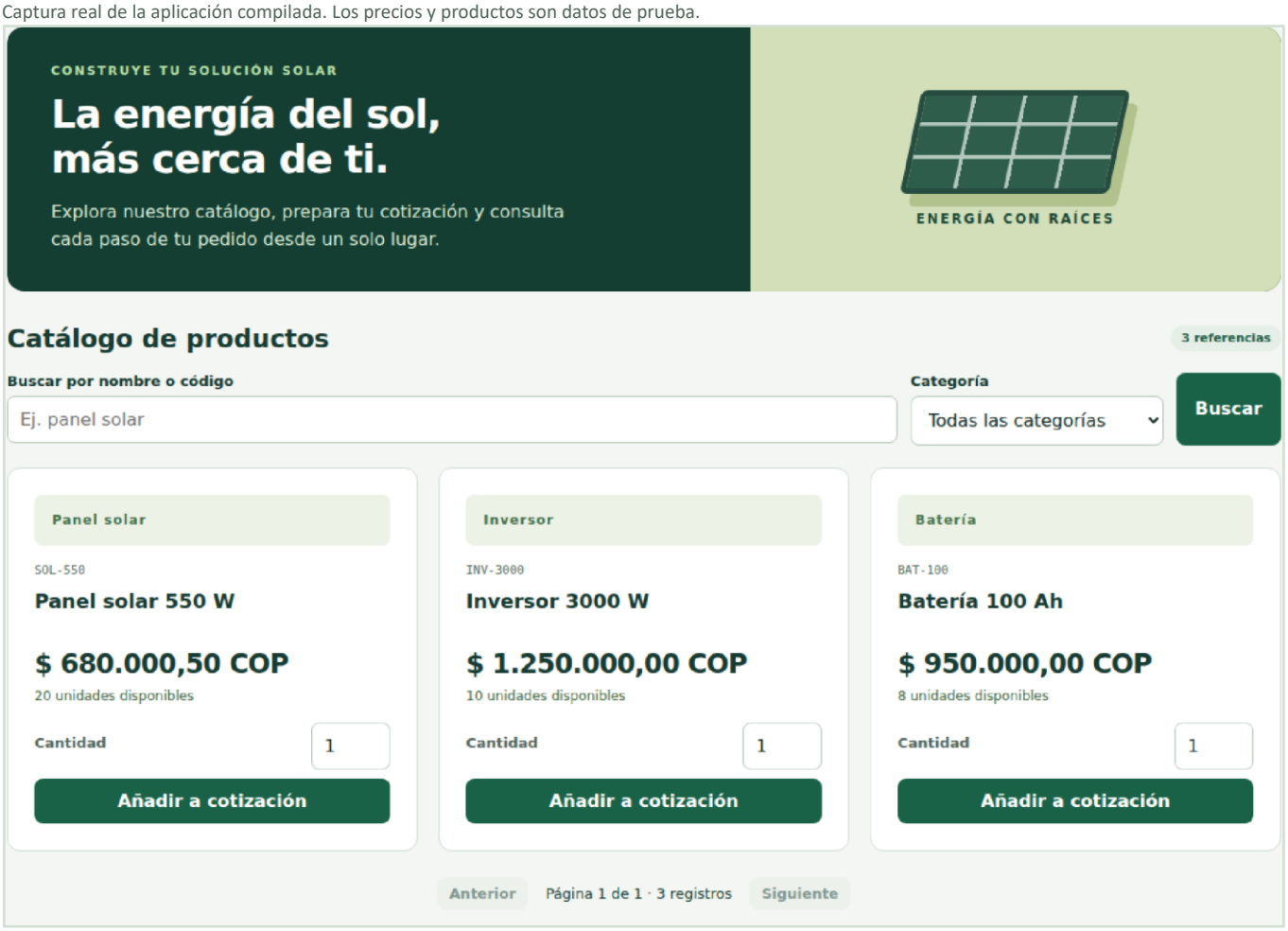


Figura 1. Catálogo conectado a la API: tres referencias activas, búsqueda, categoría, existencias y selección de cantidades. La prueba E-01 filtró “Batería” y obtuvo un único resultado.

Adaptación a dispositivos móviles

E-11 abrió el catálogo con un viewport de 390 × 844 y comprobó que no existiera desbordamiento horizontal. El diseño cambia a una columna y mantiene campos etiquetados y botones visibles. La captura completa 07_movil.png está incluida con las demás evidencias.

Las capturas son registros de ejecución; no son prototipos ni resultados inventados. En el informe se muestran detalles recortados para facilitar su lectura, y se conservan los PNG originales completos en docs/evidencias/capturas/.

Evidencia visual: venta

El cliente confirma una cotización y el servidor registra el pedido con sus datos de entrega.



Figura 2. Historial de cotizaciones con precio del servidor, cantidades y acción de confirmación. La cotización conserva el precio original.



Figura 3. Pedido pendiente: dos paneles por $1.360.001,00 COP. Las pruebas verificaron stock de 20 a 18 unidades, reintento sin descuento doble y cancelación que devolvió el saldo a 20.

La prueba E-10 confirmó después un pedido nuevo, lo marcó ENVIADO y luego ENTREGADO con ADMIN. La captura 06_pedido_entregado.png conserva ese resultado.

Evidencia visual: administración

Solo el rol ADMIN puede crear, editar, retirar productos y ajustar existencias.



Figura 4. El administrador creó KIT-E2E, editó su nombre y aumentó su stock de 5 a 8 mediante un ajuste de +3.



Figura 5. Movimientos registrados: inventario inicial, ajuste, reservas de pedidos y devolución por cancelación. E-09 verificó luego la baja lógica del producto de prueba.

Versionamiento y entrega

Se utilizó Git para conservar una base y un incremento de integración. El código local verificado está en el commit **b594bad**, precedido por **3644e05**. El ZIP incorpora un bundle que permite recuperar el historial, además del árbol de archivos listo para usar.

```
git clone versionamiento/RaizVolt_GA8.bundle RaizVolt-Git
cd RaizVolt-Git git log --oneline git status
```

Los comandos anteriores se ejecutan desde la carpeta del proyecto extraído. El bundle no contiene dependencias npm, bases de datos de usuario ni credenciales.

Repositorio y URL

Repositorio existente: https://github.com/Joan-dot650/Programa-Git

Rama de la evidencia: Joan-dot650/Programa-Git at evidencia-ga8-aa1-ev01

Base previa consultada: API-PROYECTO de AA5-EV03, commit 1a1ebb5.

Contenido de la entrega

PRODUCTO SOLICITADO	ARCHIVO / UBICACIÓN
Código fuente	src/, public/, scripts/, package.json y package-lock.json.
Archivos compilados	dist/ con servidor, interfaz, utilidades y mapas de fuentes.
Documentos	Este PDF; README.md; AMBIENTES.md; TRAZABILIDAD.md; servicios, diagramas y pruebas.
URL	Interfaz local y contrato documentados; repositorio existente identificado. Estado de publicación GA8 indicado arriba.

Lista de comprobación y cierre

Autoevaluación de la entrega frente al instrumento adjunto. La calificación y el juicio de valor corresponden al instructor.

INDICADOR DEL INSTRUMENTO	SUSTENTO	ESTADO
1. Codifica módulos según requisitos en el lenguaje seleccionado.	JavaScript, interfaz y servicios integrados; matriz RF01-RF09 y recorridos E2E.	Implementado
2. Aplica buenas prácticas de codificación.	Capas, módulos ES, validaciones, nombres claros, componentes reutilizables y comentarios.	Implementado
3. Maneja adecuadamente el IDE.	Guía de VS Code, launch.json y tareas. Se puede reproducir la depuración indicada.	Preparado para demostración
4. Aplica control de versiones.	Commits y bundle Git local. Publicación remota GA8: exitosa.	Git local verificado
5. Aplica pruebas unitarias.	28 casos unitarios; 85 pruebas Node en total y 13 comprobaciones de navegador.	Verificado

Conclusiones

Los módulos se integraron en un flujo web coherente: el catálogo alimenta cotizaciones, el perfil aporta datos de entrega y la confirmación de pedidos modifica el inventario de forma atómica. Los permisos se aplican en el servidor y los componentes compartidos evitan duplicar la lógica de comunicación y validación visual.

La compilación y las pruebas entregan un resultado reproducible. Se incluyen instrucciones para abrir la aplicación, depurar sus servicios, repetir las pruebas y recuperar el historial Git. Las limitaciones de despliegue y de verificación se mantienen explícitas para que el informe describa únicamente lo ejecutado.

Fuentes y documentación

Documentos del proyecto consultados: instrumento IE-GA8-220501096-AA1-EV01.pdf, código y diseño de API-PROYECTO incluidos en AA5-EV04, contrato OpenAPI y pruebas de la entrega anterior. Los requisitos operativos de la presente evidencia se derivan de esos archivos y de las instrucciones aportadas por el aprendiz.

1. Node.js 24.22.1 - SQLite
https://nodejs.org/download/release/v24.22.1/docs/api/sqlite.html Conexión embebida, sentencias y parámetros.

2. Node.js 24.22.1 - Test runner
https://nodejs.org/download/release/v24.22.1/docs/api/test.html Ejecución de pruebas, dobles y reportes.

3. esbuild - APIhttps://esbuild.github.io/api/
Transformación, empaquetado, plataformas y mapas de fuentes.

4. Visual Studio Code - Node.js debugginghttps://code.visualstudio.com/docs/nodejs/nodejs-debugging Configuración de lanzamiento y depuración del servidor.

5. OWASP - Password Storage Cheat Sheet
https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html Almacenamiento de contraseñas y parámetros de derivación.

6. Git - git-bundle
https://git-scm.com/docs/git-bundle
Transporte y recuperación del historial versionado.

7. Playwright - Library https://playwright.dev/docs/library
Automatización de recorridos en navegador.

Consulta técnica: 6 de noviembre de 2026. Se documentan las versiones utilizadas en la ejecución; no se afirma que sean las versiones más recientes.

Archivos que permiten auditar el resultado

docs/evidencias/PRUEBAS.tap y PRUEBAS.xml: resultados de Node.
docs/evidencias/E2E.json: recorridos y entorno del navegador.
docs/evidencias/capturas/: siete capturas originales. dist/BUILD.json: metadatos de construcción.
SHA256SUMS.txt: integridad de los archivos incluidos en el paquete.